

Lab Sheet 02

Title: Over-The-Air (OTA) Firmware Updates with ESP32

Aims: To introduce the concept, importance, and implementation of Over-The-Air (OTA) updates on the ESP32 platform. Students will learn how to enable wireless firmware updates, perform updates remotely and handle OTA events in IoT projects.

Tasks:

- Understand the OTA process and its benefits (remote maintenance, no physical access needed).
- Implement OTA using the ArduinoOTA library.
- Monitor and handle OTA events (start, progress, end, errors).
- Perform version updates and verify success.
- Explore visual feedback.

Key Concepts

- OTA: Firmware update delivered wirelessly (usually over Wi-Fi).
- Partitions: ESP32 uses separate OTA partitions to safely swap between old and new firmware.
- Workflow: Device connects to Wi-Fi → Listens for update → Downloads new binary → Validates → Reboots into new firmware.
- Advantages: Easy field updates, scalability for multiple devices.
- Risks: Interrupted updates, incompatible firmware, security vulnerabilities.

Exercise 1: Basic OTA Setup with ArduinoOTA

1. Create a new ESP32 project in Wokwi or open in Arduino IDE.
2. Connect a LED with a 220Ω current-limiting resistor to GPIO 2 (Pin 2) of the ESP32.
3. Write a sketch that connects to Wi-Fi and enables ArduinoOTA.
4. Include proper error handling and event callbacks.
5. Upload the initial version (call it Version 1) and observe the Serial Monitor.

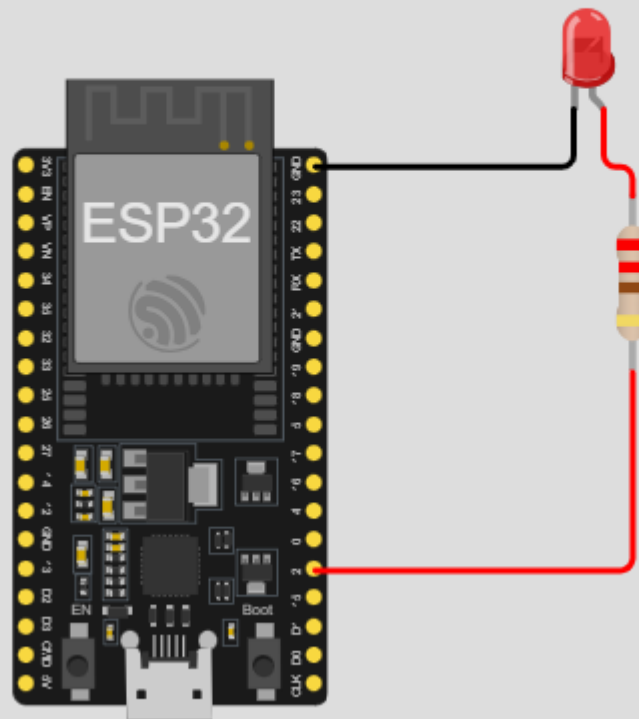
Expected Output in Serial Monitor:

Booting message

Wi-Fi connection status

IP address

OTA ready confirmation



```

#include <Arduino.h>

#include <WiFi.h>
#include <ArduinoOTA.h>

// — Configuration —————
const char* WIFI_SSID      = "Kalara-WiFi"; // Replace with your Wi-Fi SSID
const char* WIFI_PASSWORD = "Pasa2001#";   // Replace with your Wi-Fi password

const char* FIRMWARE_VERSION = "1.0.0";
const char* HOSTNAME          = "ESP32-OTA-v1";

const int LED_PIN          = 2;           // Built-in LED / GPIO 2 via 220Ω resistor
const int BLINK_DELAY      = 1000;       // 1 second blink interval (V1 pattern)

void printBanner();
void connectToWiFi();
void setupOTA();
void blinkLED();

// — Setup —————
void setup() {
    Serial.begin(115200);
    delay(500);
    pinMode(LED_PIN, OUTPUT);

    printBanner();
    connectToWiFi();
    setupOTA();

    Serial.println("✅ System ready. Waiting for OTA updates...");
    Serial.println("—————");
}

// — Loop —————
void loop() {
    ArduinoOTA.handle(); // Must be called in every loop iteration

    blinkLED();           // V1: slow blink (1 second on/off)
}

// — Wi-Fi Connection —————
void connectToWiFi() {
    Serial.printf("\n🔄 Connecting to Wi-Fi: %s\n", WIFI_SSID);
    WiFi.mode(WIFI_STA);

```

```

WiFi.begin(WIFI_SSID, WIFI_PASSWORD);

int attempts = 0;
while (WiFi.status() != WL_CONNECTED) {
    delay(500);
    Serial.print(".");
    attempts++;
    if (attempts > 40) {
        Serial.println("\n❌ Wi-Fi connection failed! Restarting...");
        ESP.restart();
    }
}

Serial.println("\n✅ Wi-Fi Connected!");
Serial.printf("    SSID      : %s\n", WiFi.SSID().c_str());
Serial.printf("    IP Address: %s\n", WiFi.localIP().toString().c_str());
Serial.printf("    Signal    : %d dBm\n", WiFi.RSSI());
}

// — OTA Configuration —————
void setupOTA() {
    ArduinoOTA.setHostname(HOSTNAME);
    // Optional: set a password for OTA security
    // ArduinoOTA.setPassword("admin");

    // — OTA Event: Start —————
    ArduinoOTA.onStart([]() {
        String type = (ArduinoOTA.getCommand() == U_FLASH) ? "Sketch" : "Filesystem";
        Serial.println("\n🚀 OTA Update Started!");
        Serial.printf("    Update type: %s\n", type.c_str());
        digitalWrite(LED_PIN, LOW); // LED off during update
    });

    // — OTA Event: End —————
    ArduinoOTA.onEnd([]() {
        Serial.println("\n✅ OTA Update Complete! Rebooting...");
        Serial.println("—————");
    });

    // — OTA Event: Progress —————
    ArduinoOTA.onProgress([](unsigned int progress, unsigned int total) {
        unsigned int percent = (progress / (total / 100));
        Serial.printf("\r📶 Uploading: [");
        int bars = percent / 5; // 20 bars total
        for (int i = 0; i < 20; i++) {

```

```

        Serial.print(i < bars ? "█" : "░");
    }
    Serial.printf("] %3u%%", percent);
});

// — OTA Event: Error —————
ArduinoOTA.onError([](ota_error_t error) {
    Serial.printf("\n❌ OTA Error [%u]: ", error);
    switch (error) {
        case OTA_AUTH_ERROR:    Serial.println("Authentication Failed"); break;
        case OTA_BEGIN_ERROR:    Serial.println("Begin Failed");          break;
        case OTA_CONNECT_ERROR:  Serial.println("Connect Failed");        break;
        case OTA_RECEIVE_ERROR:  Serial.println("Receive Failed");        break;
        case OTA_END_ERROR:      Serial.println("End Failed");            break;
        default:                 Serial.println("Unknown Error");          break;
    }
});

ArduinoOTA.begin();

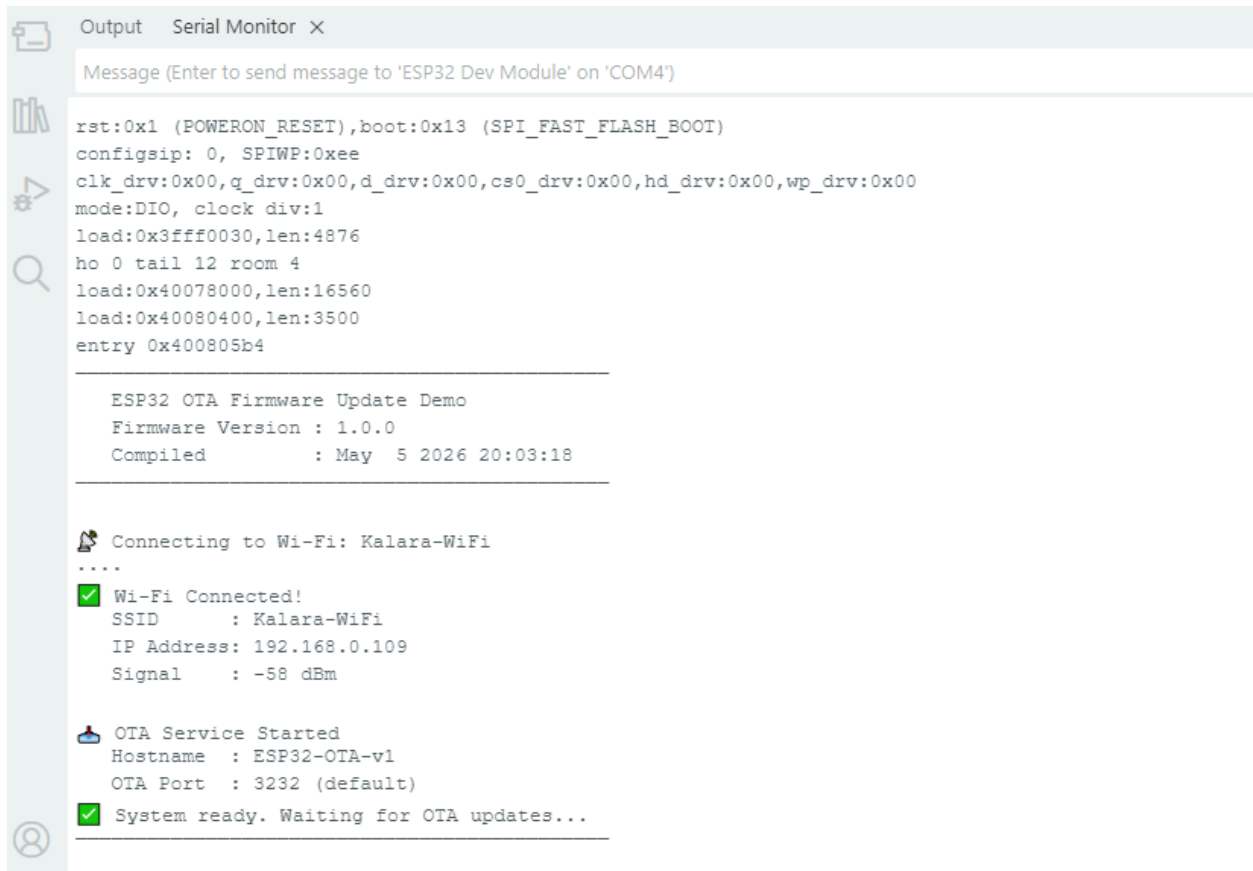
Serial.println("\n🔧 OTA Service Started");
Serial.printf("  Hostname   : %s\n", HOSTNAME);
Serial.printf("  OTA Port    : 3232 (default)\n");
}

// — LED Blink – V1 Pattern (Slow: 1s on / 1s off) —————
void blinkLED() {
    static unsigned long lastToggle = 0;
    static bool ledState = false;

    if (millis() - lastToggle >= BLINK_DELAY) {
        lastToggle = millis();
        ledState = !ledState;
        digitalWrite(LED_PIN, ledState);
    }
}

// — Banner —————
void printBanner() {
    Serial.println("—————");
    Serial.println("  ESP32 OTA Firmware Update Demo");
    Serial.printf("  Firmware Version : %s\n", FIRMWARE_VERSION);
    Serial.printf("  Compiled         : %s %s\n", __DATE__, __TIME__);
    Serial.println("—————");
}

```



The screenshot shows the Arduino IDE Serial Monitor window. The title bar reads "Output Serial Monitor X". Below the title bar is a text input field with the placeholder "Message (Enter to send message to 'ESP32 Dev Module' on 'COM4')". The main area displays the following text:

```
rst:0x1 (POWERON_RESET),boot:0x13 (SPI_FAST_FLASH_BOOT)
config: 0, SPIWP:0xee
clk_drv:0x00,q_drv:0x00,d_drv:0x00,cs0_drv:0x00,hd_drv:0x00,wp_drv:0x00
mode:DIO, clock div:1
load:0x3fff0030,len:4876
ho 0 tail 12 room 4
load:0x40078000,len:16560
load:0x40080400,len:3500
entry 0x400805b4
```

ESP32 OTA Firmware Update Demo
Firmware Version : 1.0.0
Compiled : May 5 2026 20:03:18

 Connecting to Wi-Fi: Kalara-WiFi
....
 Wi-Fi Connected!
SSID : Kalara-WiFi
IP Address: 192.168.0.109
Signal : -58 dBm

 OTA Service Started
Hostname : ESP32-OTA-v1
OTA Port : 3232 (default)

 System ready. Waiting for OTA updates...

Exercise 2: Performing Your First OTA Update

1. Modify the initial sketch to create Version 2:

- Change the version message.
- Modify LED blink pattern (e.g., faster blink or different duty cycle).
- Add a new feature (e.g., print “OTA Update Successful – Now Running v2”).

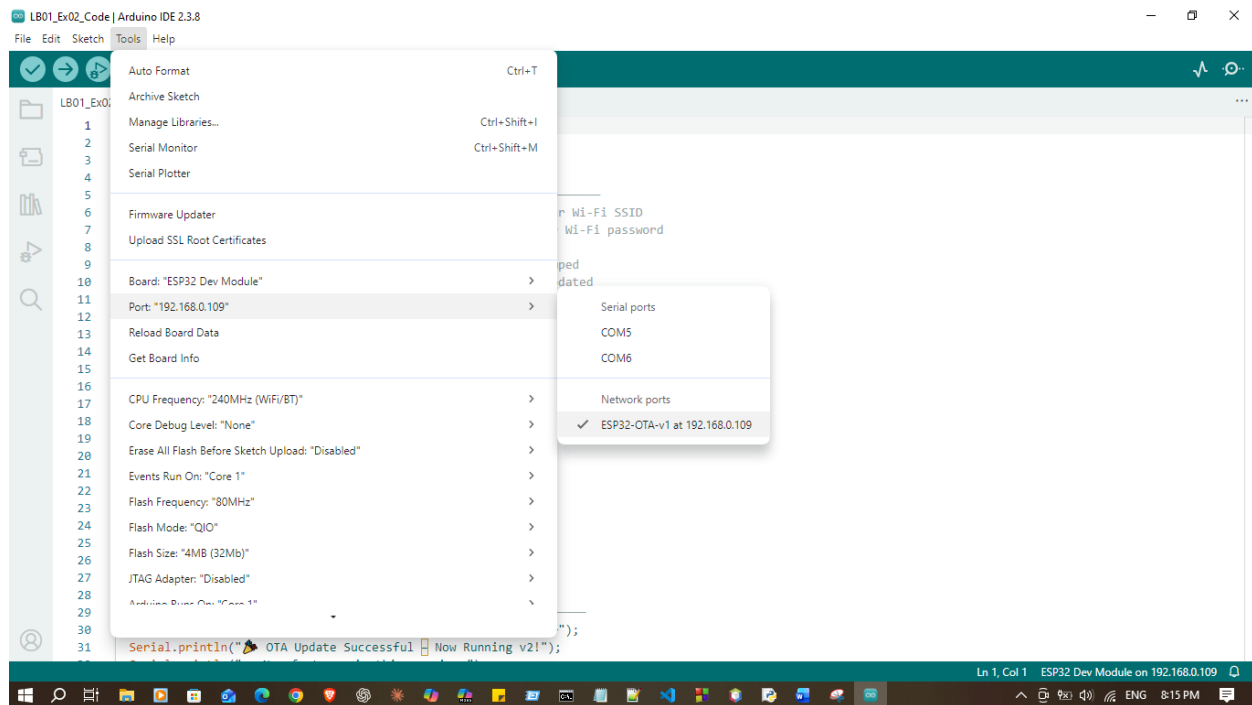
2. In Arduino IDE:

- Select the network port (ESP32's IP address).
- Upload the new sketch over-the-air.

3. Observe the update progress, completion message, and automatic reboot.

4. Verify the new behavior (LED pattern + Serial output).

Tools → Port → Select Network Port → [ESP32-OTA-v1 at 192.168.x.x] → Sketch → Upload (or Ctrl+U)



```
#include <Arduino.h>
#include <WiFi.h>
#include <ArduinoOTA.h>

// — Configuration —
const char* WIFI_SSID      = "Kalara-WiFi"; // Replace with your Wi-Fi SSID
const char* WIFI_PASSWORD = "Pasa2001#";   // Replace with your Wi-Fi password

const char* FIRMWARE_VERSION = "2.0.0";    // ← Version bumped
const char* HOSTNAME          = "ESP32-OTA-v2"; // ← Hostname updated

const int LED_PIN = 2;

// V2 Blink pattern: fast triple-flash then pause
const int FLASH_ON   = 100; // 100ms on
const int FLASH_OFF  = 100; // 100ms between flashes
const int PAUSE_TIME = 800; // 800ms pause after triple-flash

// — Function Prototypes —
void printBanner();
void connectToWiFi();
void setupOTA();
void blinkLEDv2();
```

```

// — Setup —————
void setup() {
    Serial.begin(115200);
    delay(500);
    pinMode(LED_PIN, OUTPUT);

    printBanner();
    connectToWiFi();
    setupOTA();

    // — V2 Success Message —————
    Serial.println("—————");
    Serial.println("🎉 OTA Update Successful - Now Running v2!");
    Serial.println("    New features in this version:");
    Serial.println("    • Triple-flash LED blink pattern");
    Serial.println("    • Improved OTA progress display");
    Serial.println("    • Version tracking in Serial output");
    Serial.println("—————");
}

// — Loop —————
void loop() {
    ArduinoOTA.handle();    // Must be called in every loop iteration

    blinkLEDv2();           // V2: fast triple-flash pattern
}

// — Wi-Fi Connection —————
void connectToWiFi() {
    Serial.printf("\n🔄 Connecting to Wi-Fi: %s\n", WIFI_SSID);
    WiFi.mode(WIFI_STA);
    WiFi.begin(WIFI_SSID, WIFI_PASSWORD);

    int attempts = 0;
    while (WiFi.status() != WL_CONNECTED) {
        delay(500);
        Serial.print(".");
        attempts++;
        if (attempts > 40) {
            Serial.println("\n❌ Wi-Fi connection failed! Restarting...");
            ESP.restart();
        }
    }
}

Serial.println("\n✅ Wi-Fi Connected!");

```

```

Serial.printf("    SSID      : %s\n", WiFi.SSID().c_str());
Serial.printf("    IP Address: %s\n", WiFi.localIP().toString().c_str());
Serial.printf("    Signal    : %d dBm\n", WiFi.RSSI());
}

// — OTA Configuration —————
void setupOTA() {
    ArduinoOTA.setHostname(HOSTNAME);

    // — OTA Event: Start —————
    ArduinoOTA.onStart([]() {
        String type = (ArduinoOTA.getCommand() == U_FLASH) ? "Sketch" : "Filesystem";
        Serial.println("\n🚀 OTA Update Started!");
        Serial.printf("    Update type: %s\n", type.c_str());
        Serial.printf("    Current version: %s → New version incoming...\n",
FIRMWARE_VERSION);
        digitalWrite(LED_PIN, LOW);
    });

    // — OTA Event: End —————
    ArduinoOTA.onEnd([]() {
        Serial.println("\n✅ OTA Transfer Complete!");
        Serial.println("    Validating firmware...");
        Serial.println("    Rebooting into new firmware now...");
        Serial.println("—————");
    });

    // — OTA Event: Progress —————
    ArduinoOTA.onProgress([](unsigned int progress, unsigned int total) {
        unsigned int percent = (progress / (total / 100));
        unsigned int received = progress / 1024;
        unsigned int totalKB = total / 1024;

        Serial.printf("\r📶 Uploading: [");
        int bars = percent / 5;
        for (int i = 0; i < 20; i++) {
            Serial.print(i < bars ? "█" : "░");
        }
        Serial.printf("] %3u%% (%u / %u KB)", percent, received, totalKB);
    });

    // — OTA Event: Error —————
    ArduinoOTA.onError([](ota_error_t error) {
        Serial.printf("\n❌ OTA Error [%u]: ", error);
        switch (error) {

```

```

        case OTA_AUTH_ERROR:    Serial.println("Authentication Failed - Check
password"); break;
        case OTA_BEGIN_ERROR:   Serial.println("Begin Failed - Not enough
space?"); break;
        case OTA_CONNECT_ERROR: Serial.println("Connect Failed - Network
issue?"); break;
        case OTA_RECEIVE_ERROR: Serial.println("Receive Failed - Upload
interrupted"); break;
        case OTA_END_ERROR:     Serial.println("End Failed - Validation
error"); break;
        default:                Serial.println("Unknown
Error"); break;
    }
    Serial.println(" ⚠ Device remains on current firmware.");
});

ArduinoOTA.begin();

Serial.println("\n 🚀 OTA Service Started");
Serial.printf("  Hostname  : %s\n", HOSTNAME);
Serial.printf("  OTA Port   : 3232 (default)\n");
}

// — LED Blink - V2 Pattern (Triple-flash then pause) —————
void blinkLEDv2() {
    static unsigned long lastAction = 0;
    static int step = 0;

    unsigned long now = millis();

    // Step sequence: ON, OFF, ON, OFF, ON, PAUSE (repeat)
    // 0: flash 1 ON   → FLASH_ON
    // 1: flash 1 OFF  → FLASH_OFF
    // 2: flash 2 ON   → FLASH_ON
    // 3: flash 2 OFF  → FLASH_OFF
    // 4: flash 3 ON   → FLASH_ON
    // 5: pause (LED off) → PAUSE_TIME

    const int delays[] = {
        FLASH_ON, FLASH_OFF,
        FLASH_ON, FLASH_OFF,
        FLASH_ON, PAUSE_TIME
    };
    const bool states[] = {
        HIGH, LOW,

```

```

    HIGH, LOW,
    HIGH, LOW
};

if (now - lastAction >= (unsigned long)delays[step]) {
    lastAction = now;
    digitalWrite(LED_PIN, states[step]);
    step = (step + 1) % 6;
}
}

// ——— Banner ———
void printBanner() {
    Serial.println("—————");
    Serial.println("    ESP32 OTA Firmware Update Demo");
    Serial.printf("    Firmware Version : %s\n", FIRMWARE_VERSION);
    Serial.printf("    Compiled          : %s %s\n", __DATE__, __TIME__);
    Serial.println("—————");
}

```

Enter Wi-Fi Password

